Sintaxis del lenguaje_II - Tipos No Primitivos

Los tipos de datos no primitivos (compuestos) en Rust se construyen a partir de los tipos primitivos o son definidos por el usuario.

Estos tipos ofrecen estructuras más complejas y flexibilidad en la programación.

String

Una cadena es un fragmento de texto, una colección de caracteres delimitados por “ ”

String Slices (&str)

- Representa una referencia a una secuencia de caracteres inmutables
- Es una vista (slice de una cadena), generalmente en forma de literales de cadena.

Hasta ahora hemos usado literales de cadena como “Hola Mundo” que son conocidos en tiempo de compilación.

```
let saludo : &str = "Hola Mundo";
```

String (String)

- Representa una cadena de caracteres mutable y en propiedad
- Se almacena en el **heap** que permite crecimiento dinámico
- No es un primitivo, está definido en la biblioteca estándar.

```
    let mut nombre = String::from("Jorgito");
    nombre.push_str(" Donnal");
    println!("El nombre es: {nombre}");
```

```rust
    let mut departamento = String:: new(); //se declara un string vacío
    departamento.push_str("Administración");
    println!("El departamento es: {departamento}");
```

Documentación → https://doc.rust-lang.org/std/string/struct.String.html

Métodos para cadenas

Estas opciones producen el mismo resultado:

```rust
let a = "hola".to_owned();
let b = "hola".to_string();
let c = String::from("hola");
let d: String = "hola".into();
```

Creación

```rust
let mut s1 = String::new();          // vacío
let s2 = String::from("hola");
let s3 = "hola".to_string();  //pasa a String cualquier tipo
let num = 42.to_string();             // número -> String

let s4 = "hola".to_owned(); //pasa str a String ,  guarda en el head, ahora es mutable y dueño
let s5 = String::with_capacity(50);       // reserva capacidad
```

conversión

```rust
let num = 42.to_string(); // número -> String
let n: i32 = "42".parse().unwrap();       // String -> número
```

Modificación (requiere `mut`)

```rust
let mut s = String::from("hola");
s.push('!');                    // añade un char
s.push_str(" mundo");           // añade un &str
s.insert(0, '¡');               // inserta char en posición (en bytes)
s.insert_str(1, "ey, ");        // inserta &str

// ERROR al insertar &str en byte 1 ocupado por ! que ocupa 2 bytes
// ⚠ Los índices son en bytes: paniquea si cortas en medio de un char UTF-8

s.pop();              // quita y devuelve el último char (Option<char>)
s.remove(0);          // quita el char en un índice
s.truncate(5);        // corta a N bytes
s.clear();            // vacía el string
s.retain(|c| c != 'a');    // conserva solo los chars que cumplan
```

Concatenación

```
let a = String::from("Hola");
let b = String::from(" mundo");
let c = a + &b;                    // `a` se mueve, ya no se puede usar
let d = format!("{}{}", c, "!");       // no mueve nada
let mut e = String::new();
e += "texto";              // equivale a push_str
```

Consulta

```
let s = "Hola mundo";
s.len();                // longitud en BYTES (no chars)
s.chars().count();        // número de caracteres
s.is_empty();
s.contains("mundo");       // bool
s.starts_with("Hol");
s.ends_with("do");
s.find("mundo");
s.rfind('o');             // desde el final
s.matches("o").count();    // cuántas veces aparece
```

Transformación

```
let s = "  Hola Mundo  ";
s.to_uppercase();         // "  HOLA MUNDO  "
s.to_lowercase();     // "  hola mundo   "
s.trim();               // "Hola Mundo"
s.trim_start();
s.trim_end();
s.replace("Hola", "Adiós"); // reemplaza todas
s.replacen("o", "0", 1);   // reemplaza solo N veces
s.repeat(3);              // repite el string
"##hola".trim_start_matches('#');
"hola.rs".strip_suffix(".rs");   // Option<&str>
"v1.0".strip_prefix("v");       // Some("1.0")
```

División e iteración

```
let s = "uno,dos,tres";
let partes: Vec<&str> = s.split(',').collect(); //también con String
                          s.splitn(2, ',').collect();  // máximo 2 partes
s.split_once(',');              // Some(("uno", "dos, tres"))
"a b  c".split_whitespace();       // ignora espacios múltiples
```

recorrido

```
for c in "hola".chars() { }        // por caracteres
for b in "hola".bytes() { }        // por bytes
for (i, c) in "hola".char_indices() { }
```

Slicing - partes

```
let s = "Hola mundo";
let hola = &s[0..4];       // "Hola"
let resto = &s[5..];       // "mundo"

let seguro = s.get(0..4);   // Option<&str>, no paniquea
```

Comparación

```
"a" == "a";
"a" <  "b";                    // orden lexicográfico
"Hola".eq_ignore_ascii_case("hOLA");
"a".cmp("b");
```

//Devuelve **Less** si la primera es menor que la segunda, **Equal** si son iguales y **Greater** si la primera es mayor

Formato

```
let nombre = "Ana";
let edad = 30;

format!("{} tiene {} años", nombre, edad);
format!("{nombre} tiene {edad} años");   // captura inline
format!("{:>10}", nombre);            // alinea a la derecha
format!("{:05}", 42);                 // "00042"
format!("{:.2}", 3.14159);            // "3.14"
format!("{:?}", nombre);              // Debug: "\"Ana\""
```

Vectores (Vec <T>)

Documentación → https://doc.rust-lang.org/std/vec/struct.Vec.html

En español → https://book.rustlang-es.org/ch08-01-vectors



- Un Vector es una colección dinámica de elementos del mismo tipo
- Se diferencian de los arrays en que **no son primitivos y permiten cambiar su tamaño** en ejecución. Se pueden añadir/borrar elementos por tanto **tiene que ser mutable**.
- Se almacenan en el heap

```
let v_nuevo: Vec<i32> = Vec::new(); //hay que indicar tipo si está vácio

let v = vec![1, 2, 3];
```

```
let mut numeros = vec![1, 2, 3, 4, 5];
numeros.push(6);
numeros.push(7);
numeros.push(8);

println!("El vector es: {:?}", numeros);



let mut nombres = Vec::new(); //no conoce el tipo y usa un genérico Vec<T>
nombres.push("Juanito");
nombres.push("Jorgito");
nombres.push("Jaimito");
let first = nombres[0];

println!("Los nombres son: {:?} y el primero es {}", nombres, first);
```

Es posible iterar (avanzar sobre los elementos de la colección uno a uno) sobre un vector con, por ejemplo, un for:

```
for nom in &nombres {
    println!("Nombre: {nom}");
}
```

Funciones principales:

```
let mut numeros = vec![1, 2, 3, 4, 5];

// AÑADIR

numeros.push(6); // al final: [1, 2, 3, 4, 5, 6]

numeros.insert(0, 10); // en una posición: [10, 1, 2, 3, 4, 5, 6]
```

```
numeros.extend([7, 8]); // varios a la vez: [10, 1, 2, 3, 4, 5, 6, 7, 8]
numeros.append(&mut vec![9]); // mueve otro vector al final (el otro queda vacío)

// ELIMINAR
let ultimo = numeros.pop(); // quita el último y lo devuelve (Option): Some(9)
```

** **Devuelve un option**

Option es un enum de la librería estándar que representa un valor que **puede existir o no**. Es la forma en que Rust sustituye al *null* de otros lenguajes.

```
enum Option<T> {
        Some(T), // hay un valor de tipo T
        None, // no hay valor
}
```

Es genérico: Option<i32>, Option<&str>, Option<String>, etc. Está incluido en el prelude, así que Option, Some y None se usan sin importar nada.

```
let quitado = numeros.remove(0); // quita por índice y lo devuelve: 10 (desplaza el resto)
numeros.swap_remove(1); // quita por índice, más rápido, pero NO mantiene el orden
numeros.retain(|&n| n != 4); // quita todos los que no cumplan la condición
numeros.truncate(3); // se queda solo con los 3 primeros
numeros.clear(); // vacía el vector

// ACCEDER
let mut v = vec![1, 2, 3, 4, 5];
println!("{}", v[0]); // acceso directo (hace panic si el índice no existe)
println!("{:?}", v.get(10)); // acceso seguro: Si el índice no existe devuelve None
println!("{:?}", v.first()); // Some(1)
println!("{:?}", v.last()); // Some(5)
```

```
Ejemplo:
    let pepperoni = String::from("Pepperoni");
    let mushroom = String::from("Mushroom");
    let sausage = String::from("Sausage");
    //las originales se mueven al vector y ya no se pueden usar
    let  pizza_toppings = vec![pepperoni, mushroom, sausage];
    //no puedo mover la cadena original porque se mueve y el vector quedaría con
propiedad parcial
    //tiene que ser con referencia
    //let pizza_slice = pizza_toppings[1]; //Error!!
    //let pizza_slice = &pizza_toppings[1]; //correcto

    let pizza_slice = &pizza_toppings[1..];
    println!("{pizza_slice:?}");
```

```
// MODIFICAR

v[0] = 100; // cambiar un valor

v.reverse(); // invertir

v.sort(); // ordenar de menor a mayor
```

⇒ sort() funciona con enteros; con f64 hay que usar → `sort_by(|a, b| a.partial_cmp(b).unwrap())`.

```
v.sort_by(|a, b| b.cmp(a)); // ordenar de mayor a menor

 v.dedup(); // quitar duplicados consecutivos


// CONSULTAR

println!("{}", v.len()); // número de elementos

println!("{}", v.is_empty()); // ¿está vacío?

println!("{}", v.contains(&3)); // ¿contiene el 3?

println!("{:?}", v.iter().position(|&n| n == 3)); // índice del 3: Some(...)


// RECORRER

for n in &v { println!("{}", n); } // solo leer
for n in v.iter_mut() { *n *= 2; } // modificar cada elemento
```

```
for (i, n) in v.iter().enumerate() { println!("{} -> {}", i, n); } }
```

Estructuras (structs)

- Permiten definir tipos de datos personalizados, definidos por el usuario, agrupando campos
- Pueden tener métodos asociados

```
 #[derive(Debug)]
#[allow(dead_code)]

struct Persona {
    nombre: String,
    edad: u8,
}

fn main() {
   let mut p = Persona {
        nombre: String::from("Ana"),
        edad: 30,
    };
    p.edad = 31;
    println!("{:?}", p); // Persona { nombre: "Ana", edad: 31 }
}
```

Rango – Struct Range

No es un tipo primitivo en Rust. Range es un struct normal con dos campos públicos, definido en (std::ops::Range)

Es un tipo que representa un intervalo de **valores consecutivos**. Se escribe con los operadores **..** y **..=**

Sintaxis	Tipo	Significado
`a..b`	`Range<T>`	de `a` hasta `b`, **sin** incluir `b`
`a..=b`	`RangeInclusive<T>`	de `a` hasta `b`, **incluyendo** `b`
`a..`	`RangeFrom<T>`	desde `a` hasta el infinito
`..b`	`RangeTo<T>`	desde el principio hasta `b` (excluido)
`..=b`	`RangeToInclusive<T>`	desde el principio hasta `b` (incluido)
`..`	`RangeFull`	todo el intervalo

Cuando escribes 1..5, Rust crea un valor de tipo Range<i32>, que internamente es:

```
pub struct Range<Idx> {
    pub start: Idx,
    pub end: Idx,
}
```

Es decir, solo guarda dos valores: el inicio (incluido) y el final (excluido).

```
    let v:[i32,5] = [10, 20, 30, 40, 50];
    let parte:&[i32] = &v[1..4]; // [20, 30, 40]
    println!("La parte es: {:?}", parte);
```

```
    let r = 1..10;
    println!("{}", r.contains(&5));  // true
    println!("{}", r.contains(&10)); // false
```

Es posible iterar sobre un rango con, por ejemplo, un for:

```
    let dia_mes = 1..31;
    println!("{dia_mes:?}");
    let dia_mes: Range<i32> = 1..=31;
    println!("{dia_mes:?}");

    for number in dia_mes {
        print!("{number}");
    }

    let letra: Range<char> = 'a'..'f';
    for l in letra {
        println!("{l}");
    }
```

No implementa el trait copy

```
    let mut r = 0..3;
    r.next(); // consume el 0
    r.next(); // consume el 1
    println!("{:?}", r); // 2..3  (el struct se modifica a sí mismo)
```

Tiene sus métodos…, por ejemplo:

```
    let r = 1..5;
    r.contains(&3); // true
    r.is_empty(); // false
    r.len(); // 4
```

Enumeraciones

Las enumeraciones pueden tener una serie de valores definidos por el usuario asociados a una variable.

```
#![allow(dead_code)]
#[derive(Debug)]
enum Direccion {
    Izquierda,
    Derecha,
}

#[derive(Debug)]
enum MovimientoJugador {
    Pasar,                      // Variante simple
    Correr(Direccion),          // Variante de tupla
    Teletransportar { x: u32, y: u32 }, // Variante de struct
}

enum Estaciones {
    Primavera,
    Verano,
    Otoño,
    Invierno,
}

fn main() {
    let m: MovimientoJugador = MovimientoJugador::Correr(Direccion::Izquierda);
    println!("En este turno: {:?}", m);
}
```

Tipos genéricos <T>



- Admiten cualquier tipo de dato
- Permite definir funciones genéricas que son reutilizables

El tipo genérico es como una caja en la que podemos guardar cualquier cosa.

Ejemplo, mismo tipo de datos:

```
#[derive(Debug)]
struct Punto<T> {
    x: T,
    y: T,
}

impl<T> Punto<T> {    // implementacion de tipos genericos

      fn coordenadas(&self) -> (&T, &T) {
        (&self.x, &self.y)
      }

      fn establecer_x(&mut self, x: T) {
        self.x = x;
      }
}

fn main() {
    let mut entero = Punto { x: 5, y: 10 };
    let flotante = Punto { x: 1.0, y: 4.0 };
    entero.establecer_x(7);

    println!("{entero:?} y {flotante:?}");
    println!("coordenadas: {:?}", entero.coordenadas());
}
```

Ejemplo, iguales o distinto tipo de datos:

```
#[derive(Debug)]
struct Punto<T, U> {
    x: T,
    y: U,
}
```



```rust
impl<T, U> Punto<T, U> {
    fn coordenadas(&self) -> (&T, &U) {
        (&self.x, &self.y)
    }

    fn establecer_x(&mut self, x: T) {
        self.x = x;
    }
}

fn main() {
    let mut entero = Punto { x: 5, y: 10 };
    let flotante = Punto { x: 1.0, y: 4.0 };
    entero.establecer_x(7);

    println!("{entero:?} y {flotante:?}");
    println!("coordenadas: {:?}", entero.coordenadas());

    let mezcla = Punto { x: 5, y: 4.0 }; //  T = i32, U = f64
    println!("{mezcla:?}");
}
```